

Lab 3: Singly Linked Lists

Algorithms and Data Structures (010153523) | Semester 1/2026

Duration: 2 hours (in-lab) | **Topic:** Node class, head/tail pointers, addFirst/addLast/removeFirst

Objectives

- Implement a generic `Node` nested class with `element` and `next` fields.
- Build a `SinglyLinkedList<E>` that maintains `head`, `tail`, and `size`.
- Implement `addFirst`, `addLast`, and `removeFirst` in $O(1)$ time.
- Traverse the list and understand why `removeLast` is $O(n)$.

Quick Reference

Node class (nested, private).

```
private static class Node<E> {
    E element;
    Node<E> next;
    Node(E e, Node<E> n) { element = e; next = n; }
}
```

addFirst. Create a new node pointing to the current head; update head.

addLast. Create a new node; update `tail.next` and `tail` (handle empty-list case).

removeFirst. Advance head to `head.next`; handle single-element case (tail becomes null).

Traversal.

```
Node<E> cur = head;
while (cur != null) { process(cur.element); cur = cur.next; }
```

Complexity. `addFirst/addLast/removeFirst`: $O(1)$. `removeLast`: $O(n)$.

Quick Experiments (Try It)

Try It 1: What happens on `removeFirst` from an empty list?

Add a guard: if `head == null`, throw `NoSuchElementException`. Call `removeFirst` twice on a one-element list. What is the state of `head` and `tail` after the first call? After the second?

Try It 2: Printing the list backwards

A singly linked list has no `prev` pointer. Write a recursive method `void printReverse(Node<E> node)` that prints the list in reverse by recursing to the end first, then printing on the way back. Call it with `head`.

In-Lab Exercises (target: 2 hours)

In-Lab Exercise 1: Implement `SinglyLinkedList<E>` (50 min)

Create the class with the following API:

- `int size()`, `boolean isEmpty()`.
- `E first()`, `E last()` — peek without removal; null if empty.
- `void addFirst(E e)`, `void addLast(E e)`.
- `E removeFirst()` — throws `NoSuchElementException` if empty.

- `String toString()` — e.g. `[A -> B -> C]`.
- Test with at least 8 add/remove calls. Verify `size()` after each.

In-Lab Exercise 2: Rotate (20 min)

Add `void rotate()` that moves the first element to the back in $O(1)$ time (no new node created — re-link only). Hint: move `head` to `tail.next`, advance `head`, update `tail`.

In-Lab Exercise 3: Search and Count (25 min)

Add:

- `boolean contains(E e)` — true if any node's element `equals(e)`.
- `int countOccurrences(E e)` — number of nodes whose element equals `e`.

Test with a list containing duplicate values.

In-Lab Exercise 4: Stack using a Linked List (25 min)

Using `SinglyLinkedList<E>`, implement `LinkedStack<E>` with: `push(E)`, `pop()`, `peek()`, `size()`, `isEmpty()`. Map top-of-stack to the head of the list so all operations are $O(1)$. Demonstrate with 5 pushes and 3 pops.

AI Collaboration

Suggested prompts:

- “Draw what happens to head and tail pointers when `addFirst` is called on an empty list.”
- “Why is `removeLast` $O(n)$ for a singly linked list but $O(1)$ for a doubly linked list?”

Verify: Ask the AI to implement `addFirst`. Compare with yours line by line and identify any edge case (empty list) the AI may have missed.

Take-Home (Paper Work). Solve the following on paper without using a computer or IDE. Hand-write your answers; submit at the start of the next lab. Goal: prepare you to trace and reason about code during the written exam.

Trace & Predict 1: Pointer trace

Draw the linked list after *each* operation. Show `head`, `tail`, and all `next` pointers.

```
SinglyLinkedList<Integer> L = new SinglyLinkedList<>();
L.addFirst(3); L.addFirst(1); L.addLast(5); L.removeFirst(); L.addLast(7);
```

What does `L.toString()` return at the end?

Write Code on Paper 1: Reverse a singly linked list in-place

Write `void reverse()` that reverses by re-linking nodes — no new nodes, no array. Use the three-pointer technique (`prev`, `curr`, `next`). Update `head` and `tail` correctly.

Draw on Paper 1: Memory diagram

Draw the heap memory diagram for a three-node list [A -> B -> C]. Show each Node as a two-slot box (**element**, **next**). Draw arrows for the **next** references, **head**, and **tail**.

Submission. Save files as `lab3_ex*.java` and submit on the course site.